

Bolt AI CLI: Production Roadmap

Architecture, Capabilities, and Engineering Milestones to Production

Phase 1: Core Foundation & Execution Engine

Establish a reliable, resilient main loop that handles dynamic LLM interaction without crashing.

1.1 Dynamic Model Router

Overview: Central routing engine managing model initialization based on the task type (reasoning, compaction, code generation).

- Multi-provider support (OpenAI, Anthropic, Google, Local/Ollama).
- Automatic retry logic for HTTP 429/502/504 errors.
- Fallback failover routing (switch providers if the primary is down).

1.2 Configuration & Secrets Management

Overview: Local, secure persistence of user preferences and API credentials.

- Stored in `~/.bolt/config.json` and `~/.bolt/.env`.
- Interactive 'bolt configure' CLI onboarding setup.
- Automatic credential masking in terminal logs and tracebacks.

1.3 Asynchronous Streaming & UX

Overview: Interactive, clean terminal feedback engine.

- Real-time token streaming with Rich markdown rendering.
- Syntax-highlighted code blocks.
- Spinners and status indicators during tool execution.

1.4 System Prompt Versioning

Overview: Decoupled, modular system instructions.

- Version-controlled prompt templates.
- Dynamic contextual prompt injection (tailoring instructions by active repo/language).

Phase 2: Tooling & Local Integration

Equip the agent with local system access to inspect, manipulate, and execute code safely.

2.1 Inbuilt Standard Tools

Overview: Native OS interaction capabilities.

- File read, write, and patch operations.
- Directory traversal and grep/regex search tools.
- Sandboxed standard shell command runner.

2.2 Modular Skills Folder

Overview: Extensible drop-in architecture for domain-specific functions.

- Dynamic runtime tool loader scanning `~/.bolt/skills/`.
- Decoupled tool definitions without core framework rebuilds.

2.3 Codebase Indexing & Navigation

Overview: Deep semantic repository understanding.

- AST parsing and cross-repo dependency mapping.
- Local symbol resolution and semantic search without dumping whole files into context.

Phase 3: Safety, State, & Memory

Ensure agent safety during autonomous actions while keeping context bounded and cost-effective.

3.1 Human-in-the-Loop (HITL) Guardrails

Overview: Interception of destructive operations.

- Mandatory [Y/n] confirmation for destructive shell commands (rm, git reset, drop).
- Safe-mode dry run options.

3.2 Token-Aware Memory Compaction

Overview: Context window budget management.

- Rolling compaction buffer using fast, low-cost models (Gemini Flash / Claude Haiku).
- Preserves foundational system rules and recent conversational turns.

3.3 Session Persistence (SQLite)

Overview: Non-ephemeral conversation history.

- Local SQLite storage in ~/.bolt/sessions.db.
- Session resumption via 'bolt resume '.

Phase 4: Advanced Orchestration

Coordinate complex multi-step workflows using distributed subagents and open protocols.

4.1 Subagent Delegation

Overview: Context isolation via worker agents.

- Spawn short-lived worker agents for narrow, context-heavy tasks.
- Pass only final distilled outputs back to the primary agent.

4.2 Model Context Protocol (MCP) Integration

Overview: Standardized connectivity to external toolkits.

- Native MCP client support.
- Connect to external databases, enterprise tools, and local dev environments.

4.3 Long-Term Vector Memory

Overview: Persistent retrieval-augmented generation.

- Optional local vector store (LanceDB/Chroma) integration.
- Retrieval of past architectural choices and user preferences.

Phase 5: Observability & Enterprise APIs

Provide visibility into token usage, diagnostic traces, and CI/CD integration hooks.

5.1 Debug Logging & Tracing

Overview: Diagnostic visibility for agent hallucination and tool issues.

- Rolling debug file in ~/.bolt/debug.log with raw JSON payloads and tool traces.

5.2 User Metrics Dashboard

Overview: Terminal-based usage analytics.

- 'bolt metrics' command showing token spend, costs, and tasks per provider.

5.3 Headless / CI Mode

Overview: Scriptable automation interface.

- '--non-interactive' flag for GitHub Actions and build pipelines.

5.4 API Layer

Overview: Headless server wrapper.

- FastAPI interface exposing CLI capabilities to web apps and IDE plugins.